C++ Object Relationships & The Liskov Substitution Principle

Composition (Strong HAS-A)

- It represents a strong HAS-A relationship
- one object owns another object
- contained object lifetime is tied to owner lifetime

Eg Aadya has heart

```
class Aadya {
public:
    Heart h;
    Aadya() { }
    ~Aadya() { }
};
```

Constructor / Destructor Execution Order

What happen is:

1. `Heart() { }`
2. `Aadya() { }`
3. `~Aadya() { }`
4. `~Heart() { }`



- members objects are constructed before owner objects
- members objects are destructed after owner destruction

Aggregation (Weak HAS-A)

- this is also HAS A relationship but contained object can exist independently of the owner object, its lifetime is not tied to owner lifetime

Eg A department has Teacher
and a teacher can exist even if a department is destroyed

implementation

```
class Teacher {
public:
    string name;
    Teacher(string name) { this->name = name; }
    void display() { cout << name; }
};

class Dept {
public:
    Teacher * teacher;
    Dept(Teacher * t) { teacher = t; }
    void showteacher() { teacher->display(); }
};

int main() {
    Teacher t("Aadya");
    {
        Dept d(&t);
        d.showteacher();
    } // department exists within this scope

    // department destroyed, teacher still exists
    t.display();
}
// O/p -> Aadya Aadya
```

- the owner class holds memory address of an existing object rather than owning a copy of it

Association (General Interaction)

Association -> a relationship where 2 independent objects interact with one another.
Eg Student studies under teacher.

Eg

```
class Teacher {
public:
    void teach() { cout << "teaching"; }
};

class Student {
public:
    void learn(Teacher &t) {
        t.teach();
        cout << "stud learning";
    }
};

int main() {
    Teacher t;
    Stud s;
    s.learn(t);
}
// O/p -> teaching stud learning
```

`Stud` interacts with teacher through a reference, `Stud` does not own teacher neither object lifetime is controlled by other.

Table: Relationships in OOP

Feature	Composition	Aggregation	Association
Type	strong HAS-A	weak HAS-A	general interaction
Ownership	strong ownership	non owning relation	no ownership
Lifetime	tied lifetime	independent	independent
Implementation	object member	non owning pointer	parameter or reference

Liskov Substitution Principle (LSP)

The Liskov Substitution Principle

Objects of a parent class should be replaceable with objects of its child classes without breaking correctness of program.

Example:
`Bird` shouldn't have `fly` function because birds like ostrich also inherits from it and then it wont be able to replace `Bird` cause it cant fly.

LSP Violation Warning:
If child requires stronger precondition then its a possible LSP violation.